

Written exam, Functional Programming

Tuesday August 13, 2019

- The exam duration is four hours
- There are four questions. To obtain full marks you must answer all the subquestions satisfactorily
- You are allowed to use books, lecture notes, lecture slides, hand-ins, solutions to assignments, calculators, computers, software, on-line resources etc. during the examination. This includes any form of device that can execute programs written in F#.
- You may **NOT** copy code found online that you yourself have not written and hand that in as your solution. To be safe stick to resources such as *F# for fun and profit* or Microsoft's documentation of .NET and the F# language.
- You are (unless otherwise instructed) allowed to use the .NET library including the modules described in the book, e.g., List, Set, Map etc.
- If a subquestion requires you to define a particular function, then you may (unless otherwise instructed) use that function in subsequent subquestions, even if you have not managed to define it. Providing the signature of the missing function will help in such cases.
- If a subquestion requires you to define a particular function, then you may (unless otherwise instructed) define as many helper functions as you want, but in any case you must define the required function so that it has exactly the type and effect that the subquestion asked for.
- The grading for all questions require functional solutions, and solutions with side effects will not be considered.
- You should hand in one ASCII .fs or .fsx file only, e.g. `SWU_FP2019_2.fs`. If you hand in a .fs-file then it should use the namespace `SWU_FP2019`. Other file formats than these two will not be accepted.
- You are strongly encouraged, but not required, to use the provided code skeleton `SWU_FP2019_2.fs` as a basis for your submission. Any helper functions that are provided in this file may also be part of your submission.
- Most functions that you need to write are present in the code skeleton. If an assignment asks that you write a function `isEven : int -> bool`, for instance, then there is nearly always a corresponding `let isEven _ = failwith "Not implemented"` in the source file. You may change these functions (changing a `let` to a `let rec` for instance) as long as their signatures correspond to those given in the assignment. In this case that could be `let isEven x = x % 2 = 0`. Be wary of polymorphic variables as notation sometimes differs and some IDEs, for instance, will write `MyType<'a when 'a : equality>` while others may write `MyType<'a> when 'a : equality`. These are identical.

You MUST include explanations and comments to support your solutions for the questions that

require them. You simply write them as comments around your code.

Your exam hand-in MUST be made by yourself and yourself only, and this holds for program code, examples, the explanation you provide for the code, and all other parts of the answers. It is illegal to make the exam answers as group work or to enlist the help of others in any way. This includes using solutions or code found online.

Your solution MUST compile. We reserve the right to fail any submission that does not meet this requirement.

1: Sum types (25%)

The following type `Sum<'a, 'b>` comprises two different kinds of values:

```
type Sum<'a, 'b> =  
| Left of 'a  
| Right of 'b
```

For example

```
let x = Left "Hello";;  
val x : Sum<string, 'a>  
  
let x = Right 42;;  
val x : Sum<'a, int>
```

Note that the polymorphic type `'a` is used as F# cannot figure out what the other type is (similarly to if you write `let lst = []` where F# cannot figure out the type of the list).

Question 1.1

- Write two values of type `Sum<int list, bool option>`, one should be defined using `Left` and the other should be defined using `Right`.
- Create a function `sumMap : ('a -> 'c) -> ('b -> 'c) -> Sum<'a, 'b> -> 'c` that given functions `f`, `g` and the sum `s` returns `f x` when `s = Left x` and `g y` when `s = Right y`.

Example

```
sumMap List.length String.length (Left [1; 2; 3]) = 3  
sumMap List.length String.length (Right "Hello World!") = 12
```

Question 1.2

We shall now consider the following declaration of type `SumColl<'a, 'b>` that supports the declaration of collections containing two kind of values

```
type SumColl<'a, 'b> =  
| Nil  
| CLeft of 'a * SumColl<'a, 'b>  
| CRight of 'b * SumColl<'a, 'b>
```

- Write a value of type `SumColl<bool list, int>` using all three constructors
- Create a recursive (but not tail recursive) function
`ofList : Sum<'a, 'b> list -> SumColl<'a, 'b>` that given a list with elements of type `Sum<'a, 'b>` creates a corresponding value of type `SumColl<'a, 'b>`.

Example

```
ofList [Left "Hello"; Right [1; 2; 3]; Left " world!!!"] =  
  CLeft ("Hello", CRight ([1; 2; 3], CLeft (" world!!!", Nil)))
```

Warning

F# may complain if you try `ofList []` without any type annotation as it cannot figure out the type of the list. Making the types explicitly by typing `ofList ([] : Sum<string, int list>)`, for example, solves this problem. For non-empty lists the types can most often be inferred from the elements of the list.

Question 1.3

Create a tail-recursive reversal function for values of type `SumColl<'a, 'b>`, that uses an accumulator (not a continuation).

Your functions must work directly on collections. You may **not**, for instance, translate them to normal lists, reverse them using `List.rev`, and then use `ofList` to retain a collection.

Example

```
reverse (CLeft ("Hello", CRight ([1; 2; 3], CLeft (" world!!!", Nil)))) =  
  CLeft (" world!!!", CRight ([1; 2; 3], CLeft ("Hello", Nil)))
```

Question 1.4

Make a non-recursive version of `ofList` from Question 1.2, called `ofList2`, where the signatures and the results of the two functions are the same.

Hint: Use one of the folding functions from the `List` library.

Question 1.5

Create a fold functions

```
foldBackSumColl :  
  ('a -> 'c -> 'c) ->  
  ('b -> 'c -> 'c) ->  
  SumColl<'a, 'b> ->  
  'c ->  
  'c
```

that loops over the collection from right-to-left (just as a standard list `foldBack`) and applies the appropriate argument function `f` or `g` depending on the kind of the individual elements.

You do not have to worry about making the function tail-recursive.

Example

Assuming `coll` is equal to

```
CLeft ("Hello", (CRight ([1; 2; 3], (CRight ([42], Nil)))))
```

```
foldBackSumColl  
  (fun s acc -> String.length s + acc)  
  (fun lst acc -> List.length lst - acc)  
  coll  
  0 =  
  7
```

Hint:

For standard lists, the following equivalence holds

```
foldBack f [x1; x2; ...; xn] acc = f x1 (f x2 (... (f xn acc) ...))
```

where `foldBack : ('a -> 'b -> 'b) -> 'a list -> 'b -> 'b`.

2: Code comprehension (25%)

For this assignment looking over the `String` and `List` libraries is useful.

Consider the following two functions

```
let f s =  
  let l = String.length s  
  let rec aux =  
    function  
      | i when i = l -> []  
      | i -> s.[i] :: aux (i + 1)  
  
    aux 0  
  
let g s =  
  s |> f |>  
  List.filter System.Char.IsLetter |>  
  List.map System.Char.ToLower |>  
  fun lst -> lst = List.rev lst
```

Question 2.1

- What are the types of functions `f` and `g` ?
- What do functions `f` and `g` do? Focus on what they do rather than how they do it.
- What would be appropriate names for functions `f` and `g` ?

Question 2.2

Create a function `f2` that behaves the same as `f` but which uses list comprehension.

Hint: This is a non-recursive one-line function.

Question 2.3

The function `g` uses the piping operator (`|>`) to join the building blocks of the function. With very minor changes the function can be rewritten to use function composition (`>>`) instead.

Write a function `g2` that behaves the same as `g` , with no uses of the `|>` -operator, but using the `>>` -operator instead.

Question 2.4

The function `f` is not tail-recursive. Explain why. To make a compelling argument you should evaluate a function call, similarly to what is done in Chapter 1.4 of HR, and reason about that evaluation. You need to make clear what aspects of the evaluation tell you that the function is tail recursive and you need to have at least one evaluation step per function call, recursive or otherwise.

Create a tail-recursive version of `f` called `fTail` using continuations.

Hint: When doing your evaluation steps, make sure that the expression of each step evaluate to the same value.

Question 2.5

The function `g` has linear complexity, but it is not as efficient as it can be as the functions `f`, `List.filter`, `List.map`, `List.rev`, and list equality all are linear operations. This means that the input needs to be traversed five times regardless of the result of the function.

Create a function `gOpt` that:

- has the same signature as `g`
- behaves the same as `g`
- at most traverses the input once
- returns a result as soon as it can when continued computations cannot change the result of the function

Hint: You will have to traverse the input from both the front and from the back simultaneously.

3: The golden ratio (25%)

The golden ratio ϕ is an irrational number that is approximately equal to 1.61803398875. Like π this number is frequently occurring in geometry and any student of architecture, regardless of culture, will be well versed in it. We will come to its uses a bit later but let's start by computing it.

A rather interesting way of computing the golden ratio in increasing precision is by using fibonacci numbers. As you may recall the fibonacci sequence is

$$fib = 1 \ 1 \ 2 \ 3 \ 5 \ 8 \ 13 \ 21 \ 34 \ 55 \ 89 \ 144 \dots$$

where every number of the series is the sum of the two previous numbers. An approximation of the golden ratio can then be computed as

$$\phi_n = \frac{fib_{n+1}}{fib_n}$$

For instance

$$\phi_0 = \frac{fib_1}{fib_0} = \frac{1}{1} = 1 \quad \phi_3 = \frac{fib_4}{fib_3} = \frac{5}{3} \approx 1.67 \quad \phi_{10} = \frac{fib_{11}}{fib_{10}} = \frac{144}{89} \approx 1.6180$$

Question 3.1

Create a function `calculateGoldenRatio : int -> float` that given an integer `n` returns the value of the golden ratio ϕ_n as approximated by

$$\frac{fib_{n+1}}{fib_n}$$

Make sure your code does not overflow the stack and has at most linear complexity.

Example

```
calculateGoldenRatio 0 = 1.0
calculateGoldenRatio 3 = 1.666666667
calculateGoldenRatio 42 = 1.618033989
```


Question 3.2

In this assignment we will generate infinite sequences of increasingly accurate versions of ϕ . It's important to note that this is **not** sequences of decimals of ϕ , but where each index `i` of the sequence contains the version of ϕ approximated by

$$\frac{fib_{i+1}}{fib_i}$$

For example, the first five elements of your sequences will look like this:

```
seq [1.0; 2.0; 1.5; 1.666666667; 1.6; ...]
```

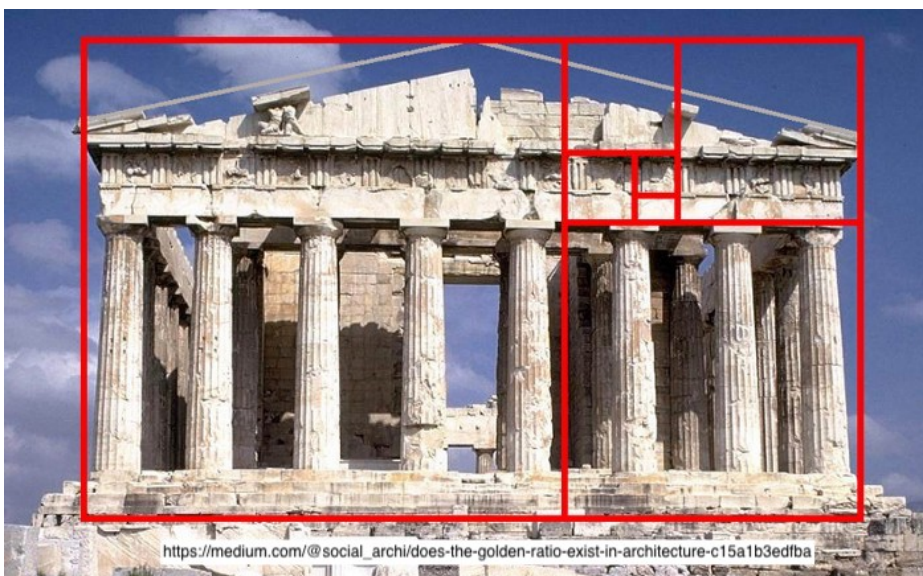
Create an infinite sequence `grSeq : seq<float>` containing the approximations of ϕ described above.

A one-line solution to this problem, which only gives partial credit, is using `Seq.infinite` and `calculateGoldenRatio` from 3.1, but this solution is not good as it recalculates the fibonacci sequences over and over again. Even though the sequence is infinite, constructing it should be linear - this means that calculating an element of the sequence should be done in constant time with the help of the data used to calculate the previous element.

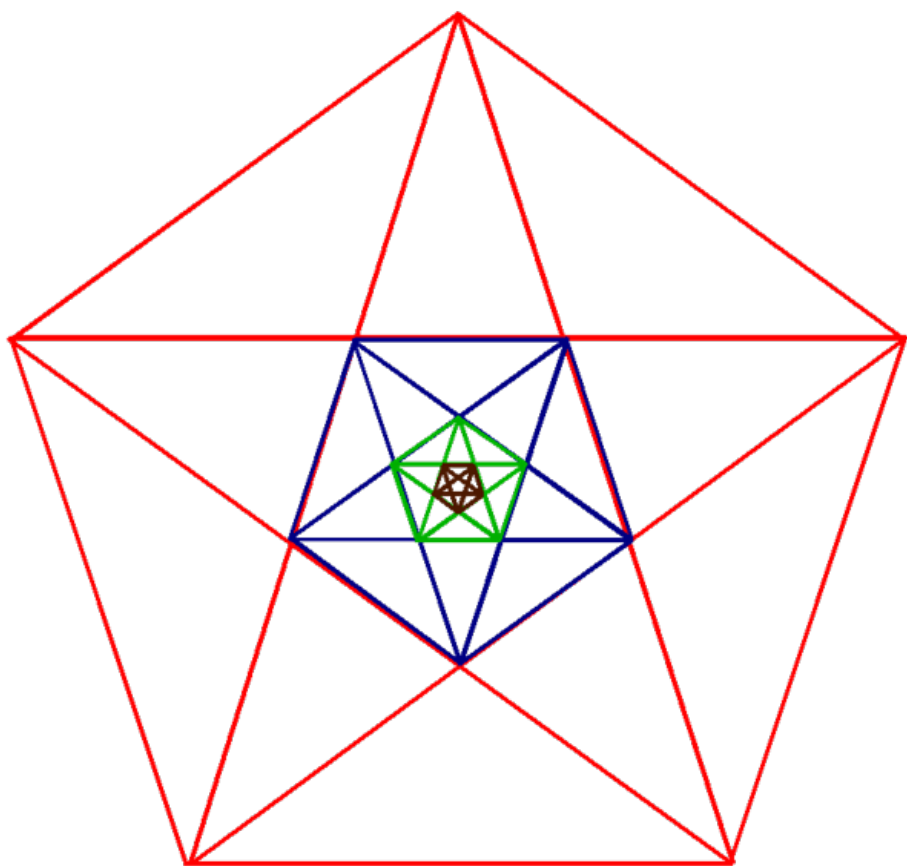
Hint Two good solutions are either using a recursive sequence expression or the `Seq.unfold` function from the `Seq` library.

Question 3.3 (Golden rectangles and triangles)

A golden rectangle is a rectangle whose sides are proportional to the golden ratio. If the length of one side is x , then the length of the longer side is ϕx . We mentioned previously that the golden rectangle can be found in architecture and the Pantheon, intentional or not, is an example of this.



A golden triangle is an isosceles triangle that has a base of length x and where the length of the two legs is ϕx . It occurs in geometry, and one example is that the tips of a pentagram (which can be created by connecting the vertices of a regular pentagon) form golden triangles.



<https://www.kisspng.com/png-golden-triangle-golden-ratio-pentagon-geometry-mat-3383625/>

Create two functions `goldenRectangleSeq : float -> seq<float>` and `goldenTriangleSeq : float -> seq<float>` that, given the length of a side `x`, returns a sequence of areas of golden rectangles and golden triangles respectively, using the approximations of ϕ from `grSeq`.

Recall that the area of a rectangle with sides that have the length a and b is ab and that the area of a triangle with base b and height h is $bh / 2$.

Calculating the height of a golden triangle with base `b` can be done using the formula.

$$b\sqrt{\phi^2 - \frac{1}{4}}$$

Example (rounded decimals for readability only)

```
goldenRectangleSeq 2.5 = seq [6.25; 12.5; 9.375; 10.4167; 10.0 ...]
goldenTriangleSeq 2.5 = seq [2.7063; 6.0515; 4.4194; 4.9684; 4.7496; ...]
```

Question 3.4

Create a function `goldenRectangleTriangle : float -> seq<float * float>` that given a floating point number `a` returns a sequence of pairs of areas of golden rectangles and triangles with side length `a` using the approximations of ϕ from `grSeq`.

For this exercise you must use sequence comprehension and **not** `Seq.zip` or any similar function.

Example

```
goldenRectangleTriangle 2.5 = seq
  [(6.25, 2.7063); (12.5, 6.0515); (9.375, 4.4194);
   (10.4167, 4.9684); (10.0, 4.7496); ...]
```

4: Qwirkle (25%)

Qwirkle is a simple little game where tiles with shapes of different colours are placed on a grid. There are six shapes - **squares**, **circles**, **stars**, **diamonds**, **clubs**, and **crosses**, and they have six colours - **red**, **green**, **blue**, **purple**, **orange**, and **yellow**. These tiles are placed on the grid such that all adjoining tiles on a row **and** column either

- have the same shape but different colours **or**
- have the same colour but different shapes

Assuming that **D** represents diamonds, **S** represents squares, **C** represents circles, then these are all valid configurations along a row or a column:

- **DDD** (same symbols, different colours)
- **DCS** (different symbols, same colours)

On the other hand, the configuration **DDD** is invalid as a blue diamond appears twice.

Valid configurations can be separated by a blank space on the grid but we will cover that later.

The full game has a scoring system as well, but for this assignment we will focus only on valid configurations.

Disclaimer For the examples bellow we will stick to the colours **blue**, **red**, and **purple** so even if you are colour blind (like the person writing this exam) you should have no difficulty differentiating between the colours. Your program must, however, support all colours.

Question 4.1

Create a type `tile` (possibly with your own auxiliary types) that represents tiles that can be placed on the board. Recall that there are six shapes - **squares**, **circles**, **stars**, **diamonds**, **clubs**, and **crosses**, and they have six colours - **red**, **green**, **blue**, **purple**, **orange**, and **yellow**.

Encoding these using infinite types like strings or integers is generally considered bad style and will not give you full credit. You may assume that the shapes and colours are fixed and you do not have to program defensively to allow them to be expanded upon at a later date.

Question 4.2

Create a function `validTiles : tile list -> tile -> bool` that given a list of valid tiles `ts` (according to the rules above), and a new tile `t` returns `true` if adding `t` to `ts` would result in a valid configuration, and false otherwise.

Example

```
validTiles [D; D] D = true
validTiles [D; D] D = false
validTiles [S; D] C = true
validTiles [S; D] S = false
```

Important: The fact that `ts` represents an already valid sequence makes your life a whole lot easier. Your function does **not** have to work if this requirement is not met.

Hint: There are functions in the `List` library that are useful here.

Question 4.3

A board is represented as a map from coordinates in two-dimensional space to your tile datatype.

```
type coord = Coord of int * int
type board = Board of Map<coord, tile>
```

If a board map has no entry for coordinate `c`, then we say that the board is empty at coordinate `c`.

Moreover, the following direction datatype comes in handy.

```
type direction = Left | Right | Up | Down
```

Create a function `moveCoord : coord -> direction -> coord` that given a coordinate `c` and a direction `d` moves the coordinate one step in direction `d` where up is decreasing on the y-axis and left is decreasing on the x-axis.

Example

```
moveCoord (Coord (0, 0)) Left  = Coord (-1, 0)
moveCoord (Coord (0, 0)) Right = Coord (1, 0)
moveCoord (Coord (0, 0)) Up    = Coord (0, -1)
moveCoord (Coord (0, 0)) Down  = Coord (0, 1)
```

Create a function `collectTiles : board -> coord -> direction -> tile list` that given a board `b`, a coordinate `c` and a direction `d` returns a list containing the tiles placed on the board `b` from coordinate `c` in direction `d` until the first empty square is encountered. The order is not important.

Example

Assuming `board` is equal to (where `#` represents a coordinate for which the board is empty)

```
01234
0 #####
1 #DDD#
2 #CCC#
3 #SSS#
4 #####
```

then (remember that your order may differ if you want)

```
collectTiles board (Coord (1, 1)) Right = DDD
collectTiles board (Coord (3, 1)) Left  = DDD
collectTiles board (Coord (1, 1)) Down  = DCS
collectTiles board (Coord (3, 3)) Up    = SCD
```

Question 4.4

Write a function `placeTile : coord * tile -> board -> board option` that given a coordinate and tile pair `(c, t)` and a valid board `b` (your function does not have to work if `b` is not valid from the start) returns `Some b'` when:

- `b'` is a valid board
- `b'` is equal to the board `b` with the tile `t` placed at coordinate `c` if that placement is valid
- the board `b` is empty at coordinate `c`

and `None` otherwise.

Example

Assuming `board` is equal to (where `#` represents a coordinate for which the board is empty)

```
0 1 2 3 4
0 #####
1 #DDD#
2 #C#C#
3 #SSS#
4 #####
```

then

```
placeTile (Coord (2, 2), C) board = Some newBoard
placeTile (Coord (2, 2), C) board = None (* duplicate colour *)
placeTile (Coord (2, 2), D) board = None (* duplicate shape *)
placeTile (Coord (1, 1), D) board = None (* occupied coordinate *)
```

where `newBoard` is equal to

```
0 1 2 3 4
0 #####
1 #DDD#
2 #CCC#
3 #SSS#
4 #####
```

Question 4.5

Create a function `placeTiles : (coord * tile) list -> board -> board option` that given a list of coordinates and tiles `ts` and a valid board `b` (your function does not have to work if `b` is not valid from the start) attempts to place all tiles from `ts` on their respective coordinates on `b` and returns `Some b'` when:

- `b'` is a valid board
- `b'` is equal the board `b` with all tiles from `ts` placed at their respective coordinates
- the board `b` is empty at all coordinates in `ts`

and `None` otherwise.

To get full marks on this assignment you must use railway-oriented programming or computation expressions (no matches on options within `placeTiles` or any other auxiliary function you create for this question). Functions from previous questions like `placeTile`, for instance, may match on options. You are not expected to go back and change those.

The option monad, or the computation expression, defined below come in handy. You may use either one and you do **not** have to use both.

Option monad

```
let ret = Some
let bind f =
  function
  | None    -> None
  | Some x -> f x

let (>>=) x f = bind f x
```

Computation expressions

```
type opt<'a> = 'a option

type OptBuilderClass() =
  member t.Return (x : 'a) : opt<'a> = ret x
  member t.ReturnFrom (x : opt<'a>) = x
  member t.Bind (o : opt<'a>, f : 'a -> opt<'b>) : opt<'b> = bind f o

let opt = new OptBuilderClass()
```


Example

Assuming `board` is equal to (where `#` represents a coordinate for which the board is empty)

```
01234
0 #####
1 #DDD#
2 #####
3 #SSS#
4 #####
```

then

```
placeTiles [] b = board
placeTiles [(Coord (2, 2), C)] board = placeTile (Coord (2, 2), C) board

placeTiles
  [(Coord (1, 2), C); (Coord (2, 2), C); (Coord (3, 2), C)] board =
  Some newBoard
placeTiles
  [(Coord (1, 2), C); (Coord (2, 2), C); (Coord (3, 2), C)] board =
  None (* duplicate colour *)
placeTiles
  [(Coord (1, 2), C); (Coord (2, 2), D); (Coord (3, 2), C)] board =
  None (* duplicate shape *)
placeTiles
  [(Coord (1, 1), C); (Coord (2, 1), C); (Coord (3, 1), C)] board =
  None (* occupied coordinate(s) *)
```

where `newBoard` is equal to

```
01234
0 #####
1 #DDD#
2 #CCC#
3 #SSS#
4 #####
```